

University of Sheffield



COM3529 Software Testing and Analysis

Coverage Criteria

Dr José Miguel Rojas

Roadmap

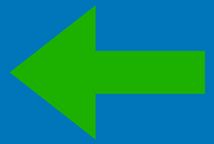
Introduction to Testing

Automation – testing at different levels

Unit Testing

Unit Isolation and Larger Tests

Coverage Criteria



Regression Testing

Mutation Testing

Model-Based Testing

Fuzzing

Search-based Test Generation

Recap from Week 1

Why Finding ALL Bugs is Impossible, or:
Why Software Testing is Hard

1. Executing all inputs for any non-trivial program is **intractable**
2. Ensuring the software will terminate with every input is **undecidable**
3. Recognising correct/incorrect outputs given their corresponding inputs is at least as hard as building the software in the first place – **the oracle problem**

Coverage Criteria

A **coverage criterion** takes an abstract representation of a piece of software and divides it up into **testable features**.

Each feature forms the basis of a **test requirement** – something that **needs to be tested by the software's test suite**.

> Not to be confused with a **software requirement** and also not the same as a test case – a test case can fulfil several test requirements.

When a test case or test suite **fulfils** a test requirement, we say the test requirement is **covered**. The percentage of test requirements covered by a test suite is then called its **coverage level** (or just its "**coverage**").

Coverage criteria are a divide-and-conquer approach to testing.

Statement Coverage

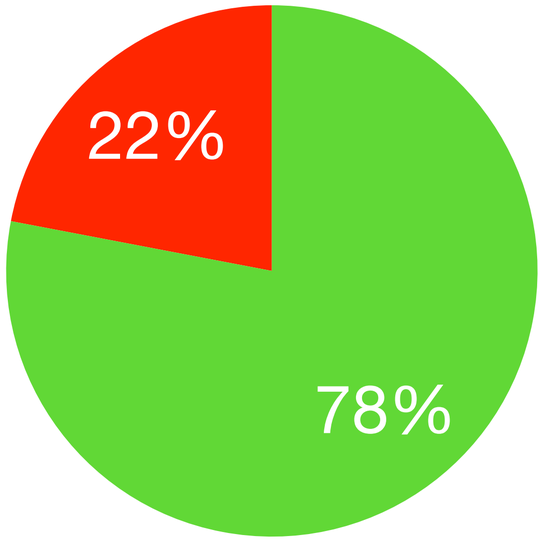
One **very simple representation** is the **program statements** that make up a piece of software.

Each **feature** is a program statement.

Each test requirement involves executing (“covering”) each line.

Statement coverage is the percentage of lines of code executed by the test suite.

Coverage Level



Some tools, e.g., JaCoCo, measure **instructions** instead of statements because they compute coverage at the bytecode level.

78% of statements executed (covered)

22% of statements not executed (uncovered)

The coverage level (or just “coverage”) is the percentage of test requirements fulfilled (executed) by the test suite.

Here, 78% statements are executed, so the **statement coverage is 78%**.

JaCoCo Coverage Report

```
1.  package uk.ac.shef.com3529.forum;
2.
3.  import java.time.Instant;
4.  import java.util.*;
5.
6.  public class Forum {
7.
8.      Set<User> users;
9.      Set<User> online;
10.     List<Post> posts;
11.
12.     public Forum() {
13.         users = new HashSet<>();
14.         online = new HashSet<>();
15.         posts = new LinkedList<>();
16.     }
17.
18.     User getUser(String username) {
19.         for (User user : users) {
20.             if (user.getUsername().equals(username)) {
21.                 return user;
22.             }
23.         }
24.         return null;
25.     }
26.
27.     User getUserThrowExceptionIfDoesNotExist(String username) {
28.         User user = getUser(username);
29.         if (user == null) {
30.             throw new UnknownUserException("Unknown user " + user);
31.         }
32.         return user;
33.     }
```

Green = **executed** by test(s)

Yellow = one ***branch*** (true or false) of an `if` or `for`/`while` statement executed by test(s)

Red = **not executed** by test(s)

Criticisms and Other Coverage Criteria

Statements are one very simple way of representing the software and dividing it up (and then “conquering”) for testing.

Statement Coverage is not that useful a method of “deciding what to test”, more a way of discovering what is *not* covered by a test suite, and deciding whether more test cases are needed.

In this lecture, we will learn different (and potentially more useful) coverage criteria, e.g., branch coverage and MCDC.

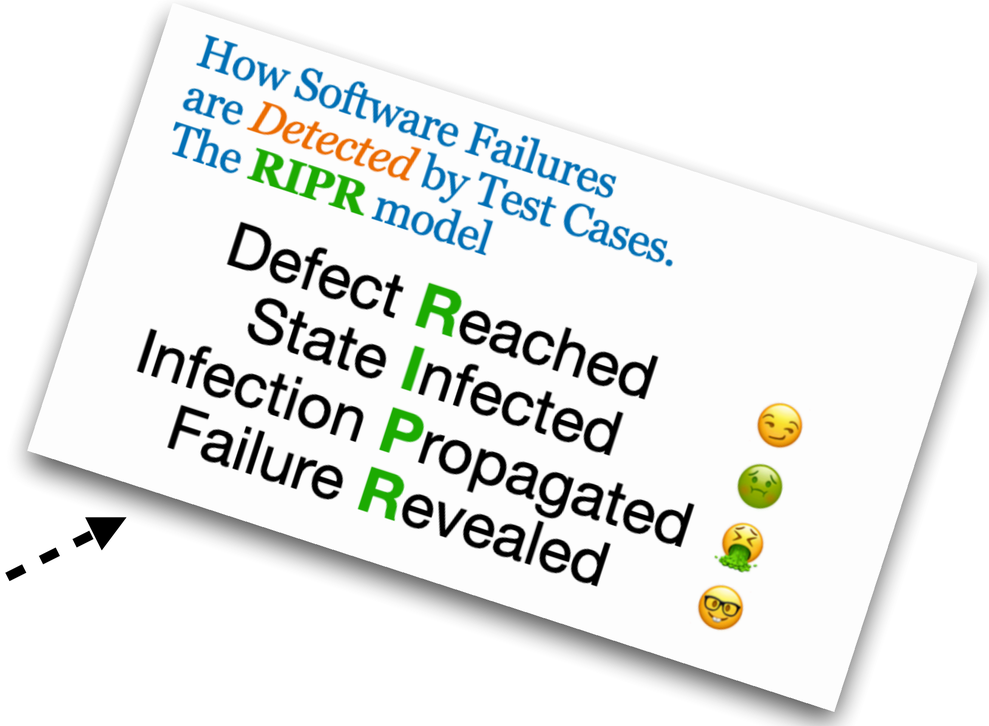
Does Obtaining 100% Coverage Ensure our Program is Bug-Free?

Does Obtaining 100% Coverage Ensure our Program is Bug-Free?

NO! – coverage criteria, such as statement coverage, are just a method of generating test requirements in a systematic way.

For example, 100% statement coverage might execute the *majority* of defects, but:

- **does not** guarantee that the defects will **infect** the program's state
- **does not** guarantee infections will **propagate** to the program’s output as failures
- **does not** guarantee failures will be **caught** by the test suite’s assertions



Recall the RIPR model from week 1

Infeasible Test Requirements

Moreover, some test requirements may be **infeasible**.

This means they are **impossible** to fulfil (i.e., cover with a test).

For example, a line of code that cannot be executed – i.e., dead code.

```
if (false) {

  aMethodCallThatNeverHappens();

}
```

This program statement can **never** be executed!

It is an **infeasible test requirement** for statement coverage.

Types of Coverage Criteria

Coverage criteria can be categorised by whether the test requirements are derived from the code itself or from more abstract artefacts.



White Box

Full knowledge of the internal code structure.
Hence often called **code coverage** criteria.

Grey Box

Some knowledge of internal code structure.
Coverage criteria based on a mix of “white box” and “black box” artefacts.



Black Box

No knowledge of the internal code structure.
Coverage criteria based on requirements, designs, or abstract models of the software.

Different Criteria, Different Defects

Defect of **commission**

Code does something *more* than it is supposed to

e.g., a C program iterating beyond the end of an array, causing a buffer overflow

Defect of **omission**

Code does something *less* than it is supposed to

e.g., an unimplemented requirement



Software



White Box Coverage Criteria

Best suited to detecting defects of commission
as can “see” into code.

Less well suited to detecting defects of omission
as no knowledge of software requirements.

Black Box Coverage Criteria

Less well suited to detecting defects of commission
as no knowledge of internal workings of the software.

Best suited to detecting defects of omission
as based on requirements, abstract models of software.

White-Box Coverage Criteria based on Control Flow Analysis

Control Flow

Realistic code under test often has a fairly **complex control flow**, with several **nested loops** and **conditional statements**, and **multiple variables** being manipulated in different ways.

We could analyse the code by hand, but this can be **very costly**, particularly as programs grow in complexity.

```
public static Set<Character> duplicateLetters(String s) {
    // lower case the string and remove all characters
    // that are not letters
    s = s.toLowerCase().replaceAll("[^a-z.]", "");

    // initialise the result set
    Set<Character> duplicates = new TreeSet<>();

    // iterate through the string
    for (int i = 0; i < s.length(); i++) {
        char si = s.charAt(i);

        // iterate through the rest of the string,
        // checking for the same letter
        for (int j = i + 1; j < s.length(); j++) {
            char sj = s.charAt(j);

            if (si == sj) {
                // a match has been found, add it to
                // the result set
                duplicates.add(si);
            }
        }
    }

    return duplicates;
}
```

Control Flow

Realistic code under test often has a fairly **complex control flow**, with several **nested loops** and **conditional statements**, and **multiple variables** being manipulated in different ways.

We could analyse the code by hand, but this can be **very costly**, particularly as programs grow in complexity.

```
public static Set<Character> duplicateLetters(String s) {
    // lower case the string and remove all characters
    // that are not letters
    s = s.toLowerCase().replaceAll("[^a-z.]", "");

    // initialise the result set
    Set<Character> duplicates = new TreeSet<>();

    // iterate through the string
    for (int i = 0; i < s.length(); i++) {
        char si = s.charAt(i);

        // iterate through the rest of the string,
        // checking for the same letter
        for (int j = i + 1; j < s.length(); j++) {
            char sj = s.charAt(j);

            if (si == sj) {
                // a match has been found, add it to
                // the result set
                duplicates.add(si);
            }
        }
    }

    return duplicates;
}
```

Control Flow

Realistic code under test often has a fairly **complex control flow**, with several **nested loops** and **conditional statements**, and **multiple variables** being manipulated in different ways.

We could analyse the code by hand, but this can be **very costly**, particularly as programs grow in complexity.

```
public static Set<Character> duplicateLetters(String s) {
    // lower case the string and remove all characters
    // that are not letters
    s = s.toLowerCase().replaceAll("[^a-z.]", "");

    // initialise the result set
    Set<Character> duplicates = new TreeSet<>();

    // iterate through the string
    for (int i = 0; i < s.length(); i++) {
        char si = s.charAt(i);

        // iterate through the rest of the string,
        // checking for the same letter
        for (int j = i + 1; j < s.length(); j++) {
            char sj = s.charAt(j);

            if (si == sj) {
                // a match has been found, add it to
                // the result set
                duplicates.add(si);
            }
        }
    }

    return duplicates;
}
```

Control Flow

Realistic code under test often has a fairly **complex control flow**, with several **nested loops** and **conditional statements**, and **multiple variables** being manipulated in different ways.

We could analyse the code by hand, but this can be **very costly**, particularly as programs grow in complexity.

```
public static Set<Character> duplicateLetters(String s) {
    // lower case the string and remove all characters
    // that are not letters
    s = s.toLowerCase().replaceAll("[^a-z.]", "");

    // initialise the result set
    Set<Character> duplicates = new TreeSet<>();

    // iterate through the string
    for (int i = 0; i < s.length(); i++) {
        char si = s.charAt(i);

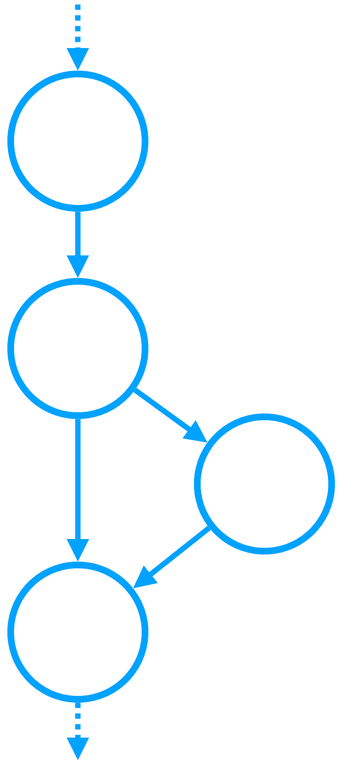
        // iterate through the rest of the string,
        // checking for the same letter
        for (int j = i + 1; j < s.length(); j++) {
            char sj = s.charAt(j);

            if (si == sj) {
                // a match has been found, add it to
                // the result set
                duplicates.add(si);
            }
        }
    }

    return duplicates;
}
```

Control Flow Graph (CFG)





```java
public static Set<Character> duplicateLetters(String s) {
    // lower case the string and remove all characters
    // that are not letters
    s = s.toLowerCase().replaceAll("[^a-z.]", "");

    // initialise the result set
    Set<Character> duplicates = new TreeSet<>();

    // iterate through the string
    for (int i = 0; i < s.length(); i++) {
        char si = s.charAt(i);

        // iterate through the rest of the string,
        // checking for the same letter
        for (int j = i + 1; j < s.length(); j++) {
            char sj = s.charAt(j);

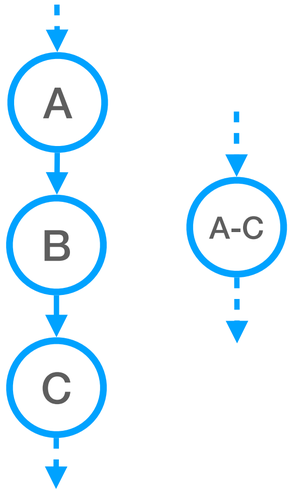
            if (si == sj) {
                // a match has been found, add it to
                // the result set
                duplicates.add(si);
            }
        }
    }

    return duplicates;
}
```

CFG – Building Blocks

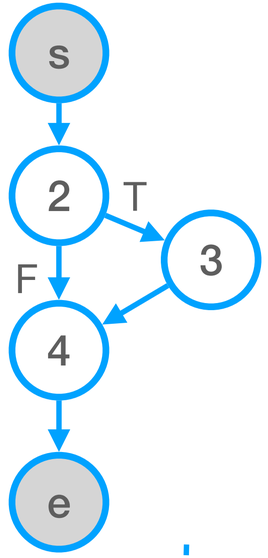
```
doSomething(); /* A */
doSomethingElse(); /* B */
doSomethingDifferent(); /* C */
```

Sequential code



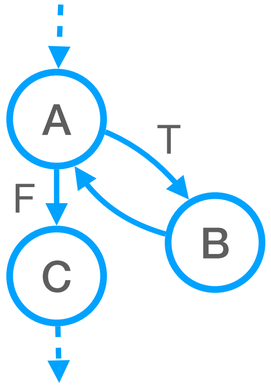

```
1 public void printGreeting() {
2   if (isMorning()) {
3     System.out.println("Good Morning!");
4   }
5 }
```

Methods and If statements



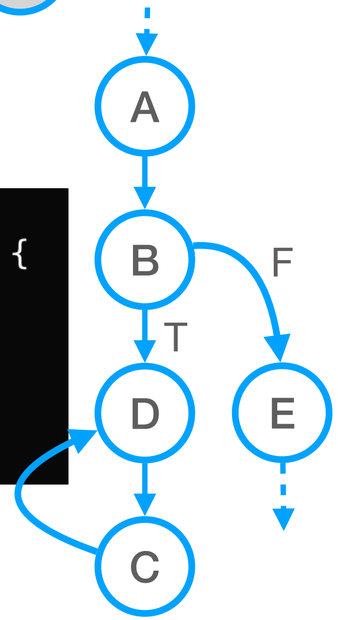

```
while (condition) { /*A*/
  doSomething(); /* B */
}
/* C */
```

While loops



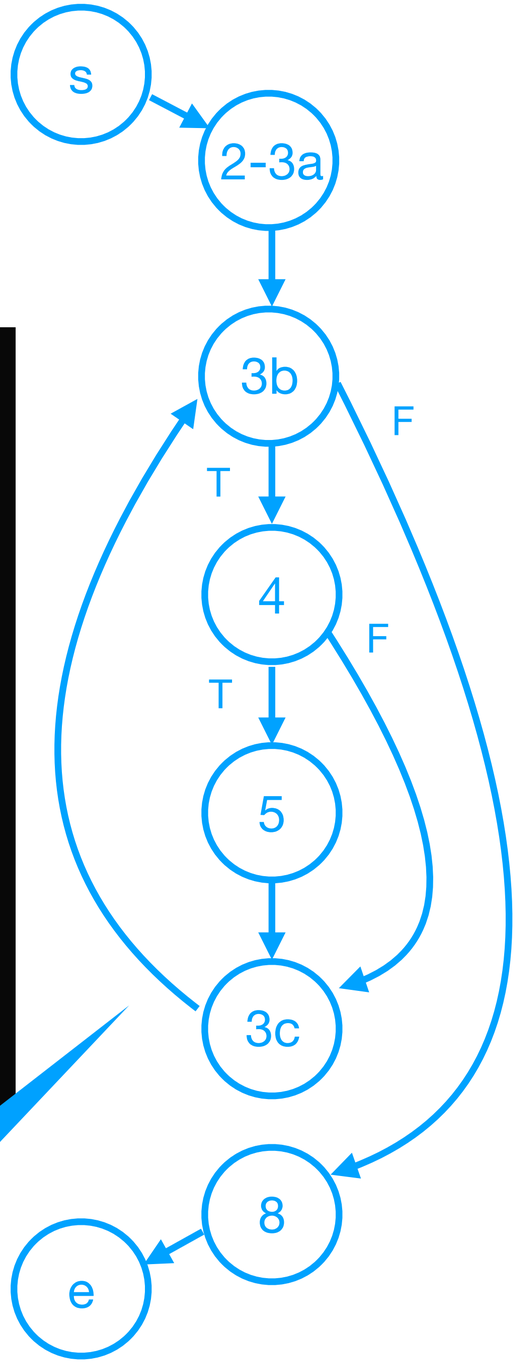

```
for (/*A*/ int i = 0; /*B*/ i < 10; /*C*/ i ++) {
  doSomething(); /* D */
}
// E
```

For loops



An Example

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



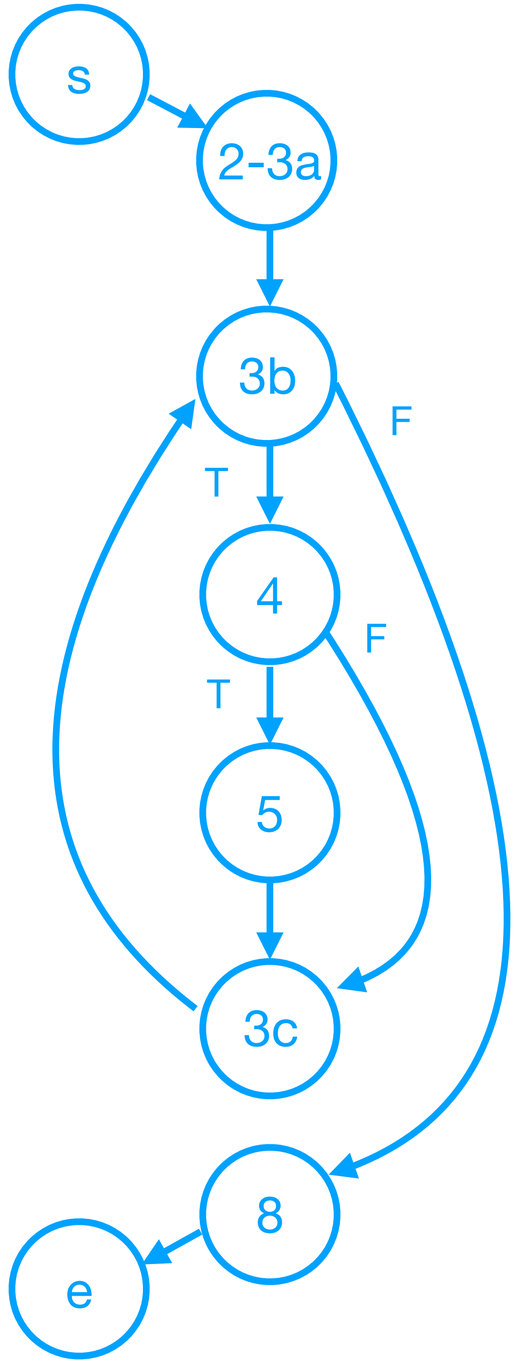
Node labels correspond to line numbers in the code.

Statement Coverage

Also known as Line Coverage, Node Coverage, and Basic Block Coverage

The test suite should execute **all nodes** of the CFG

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Statement Coverage

Also known as Line Coverage, Node Coverage, and Basic Block Coverage

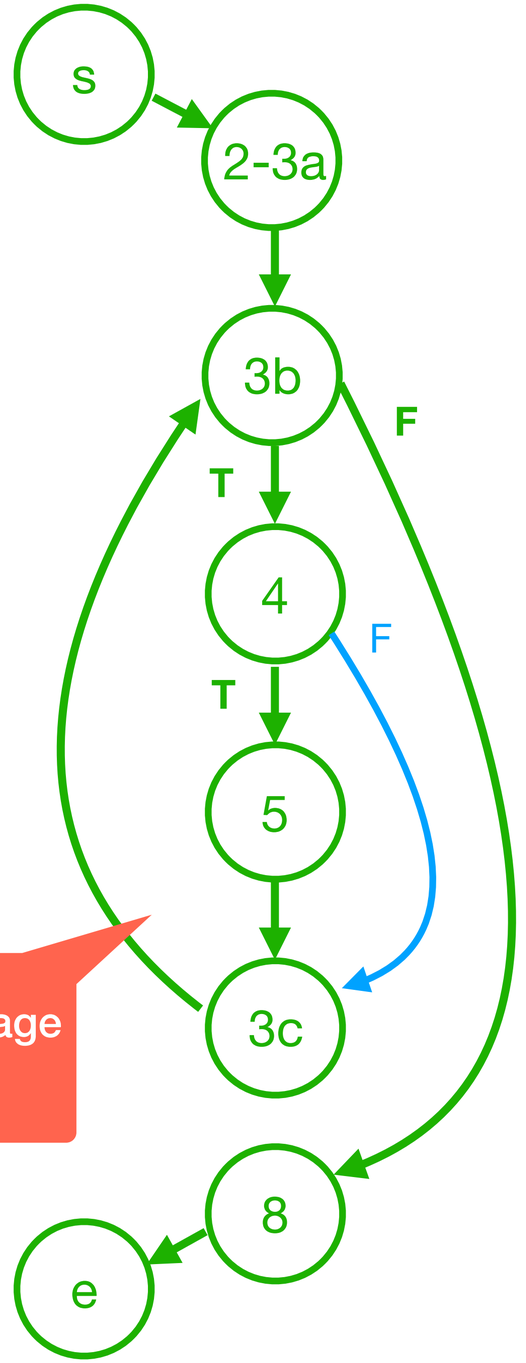
The test suite should execute **all nodes** of the CFG

A single test case **x = [0]** would execute **all nodes**:

s → 2-3a → 3b → 4 → 5 → 3c → 3b → e

In practice, covering all nodes often requires several test cases

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Is 100% statement coverage good enough?

Statement Coverage

Also known as Line Coverage, Node Coverage, and Basic Block Coverage

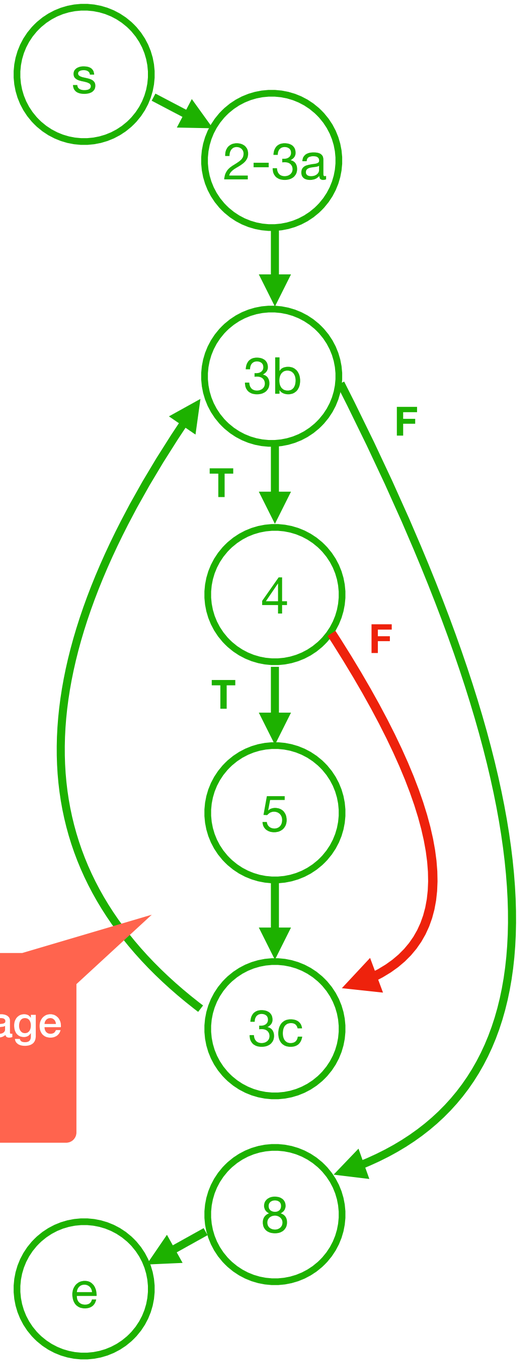
The test suite should execute **all nodes** of the CFG

A single test case **x = [0]** would execute **all nodes**:

s → 2-3a → 3b → 4 → 5 → 3c → 3b → e

In practice, covering all nodes often requires several test cases

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



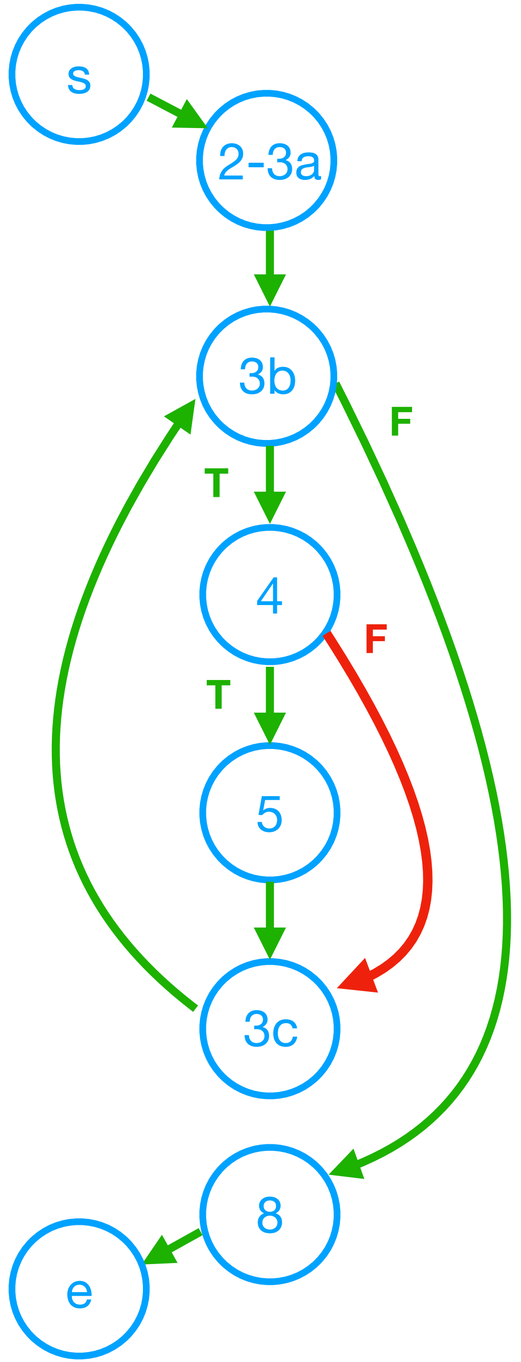
Is 100% statement coverage good enough?

Branch Coverage

Also known as Decision Coverage, Predicate Coverage, and Edge Coverage

The test suite should execute **all true/false edges** of the CFG

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Branch Coverage

Also known as Decision Coverage, Predicate Coverage, and Edge Coverage

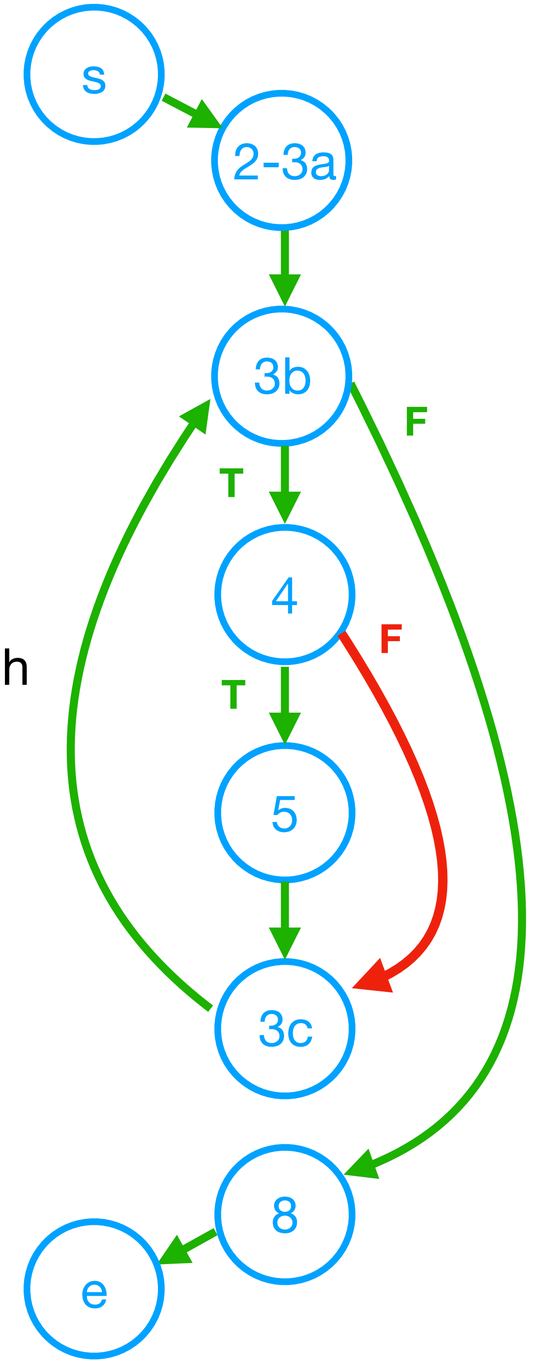
The test suite should execute **all true/false edges** of the CFG

The test cases **x = [0]** and **x = [1]** would execute all true/false branches

x = [0] takes the true branch from node 4, **x = [1]** covers the missed false branch

Note that these test cases could also be merged into one: **x = [0, 1]**

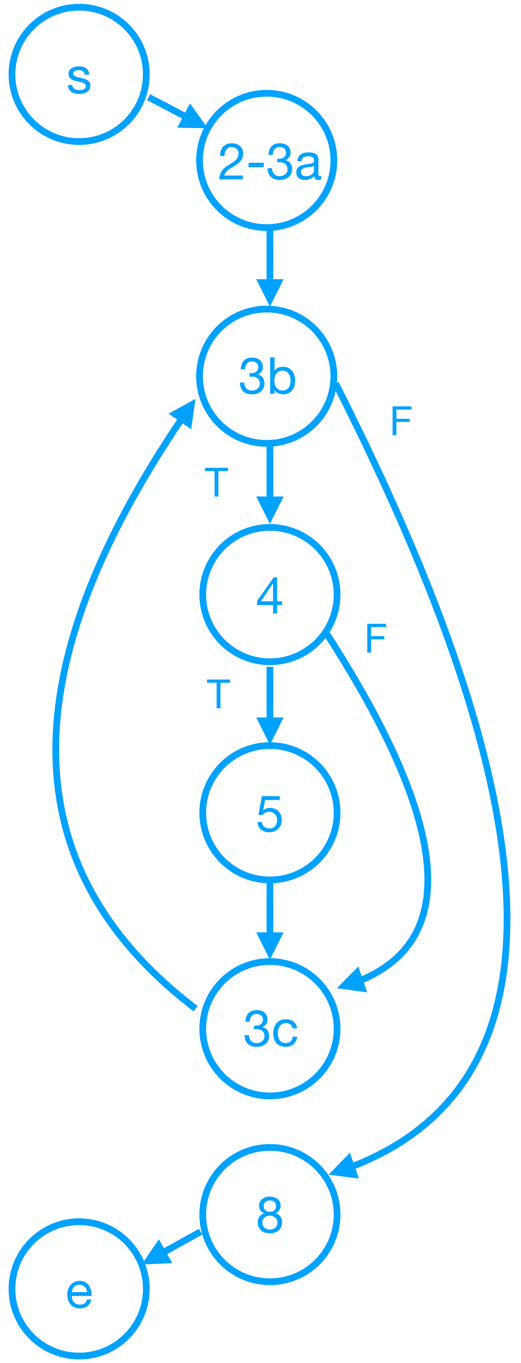
```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Path Coverage

The test suite should execute **all paths** through the CFG

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



Path Coverage

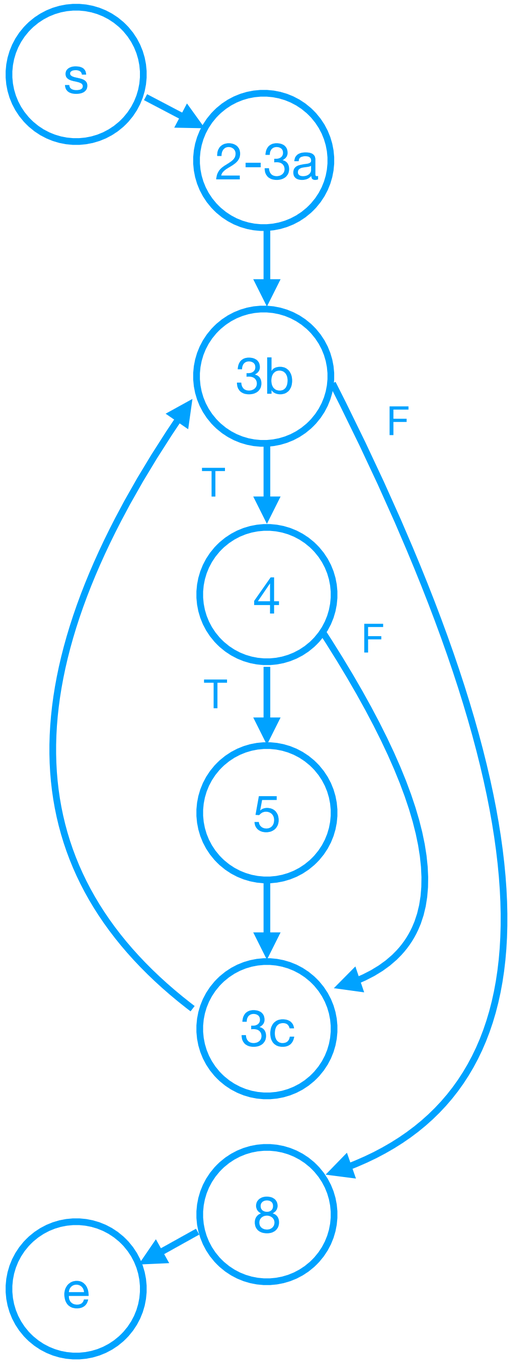
The test suite should execute **all paths** through the CFG

The number of paths through `countZeros` **depends on the length of** `x`

Usually **not possible in practice**

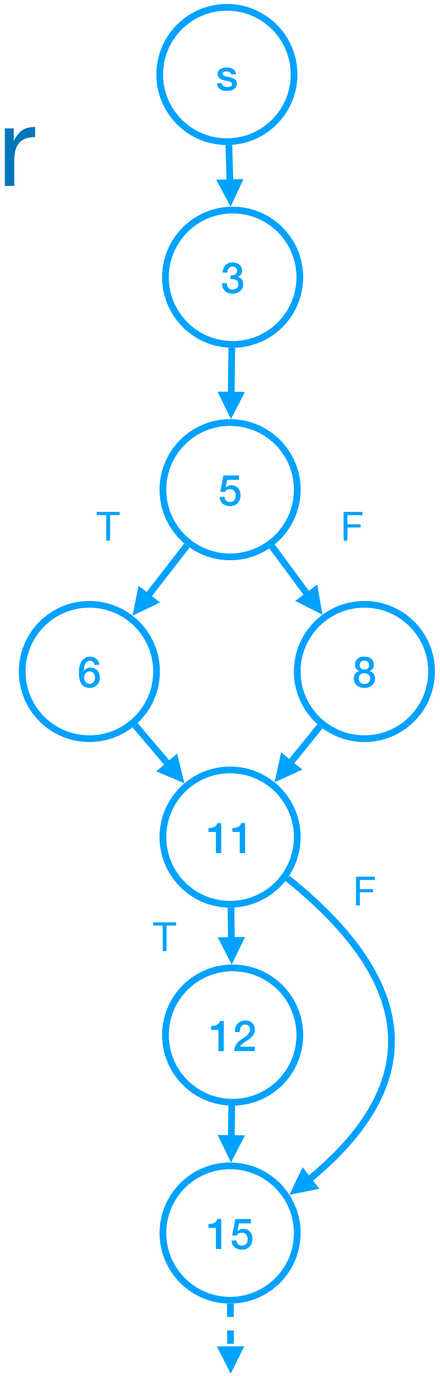
Bounded Path Coverage focuses on **0**, **1** or **N** loop executions to mitigate the **potentially infinite** number of paths

```
1 public int countZeros(int[] x) {
2   int count = 0;
3   for (int i=0 /* 3a */; i < x.length /* 3b */; i++ /* 3c */) {
4     if (x[i] == 0) {
5         count ++;
6     }
7   }
8   return count;
9 }
```



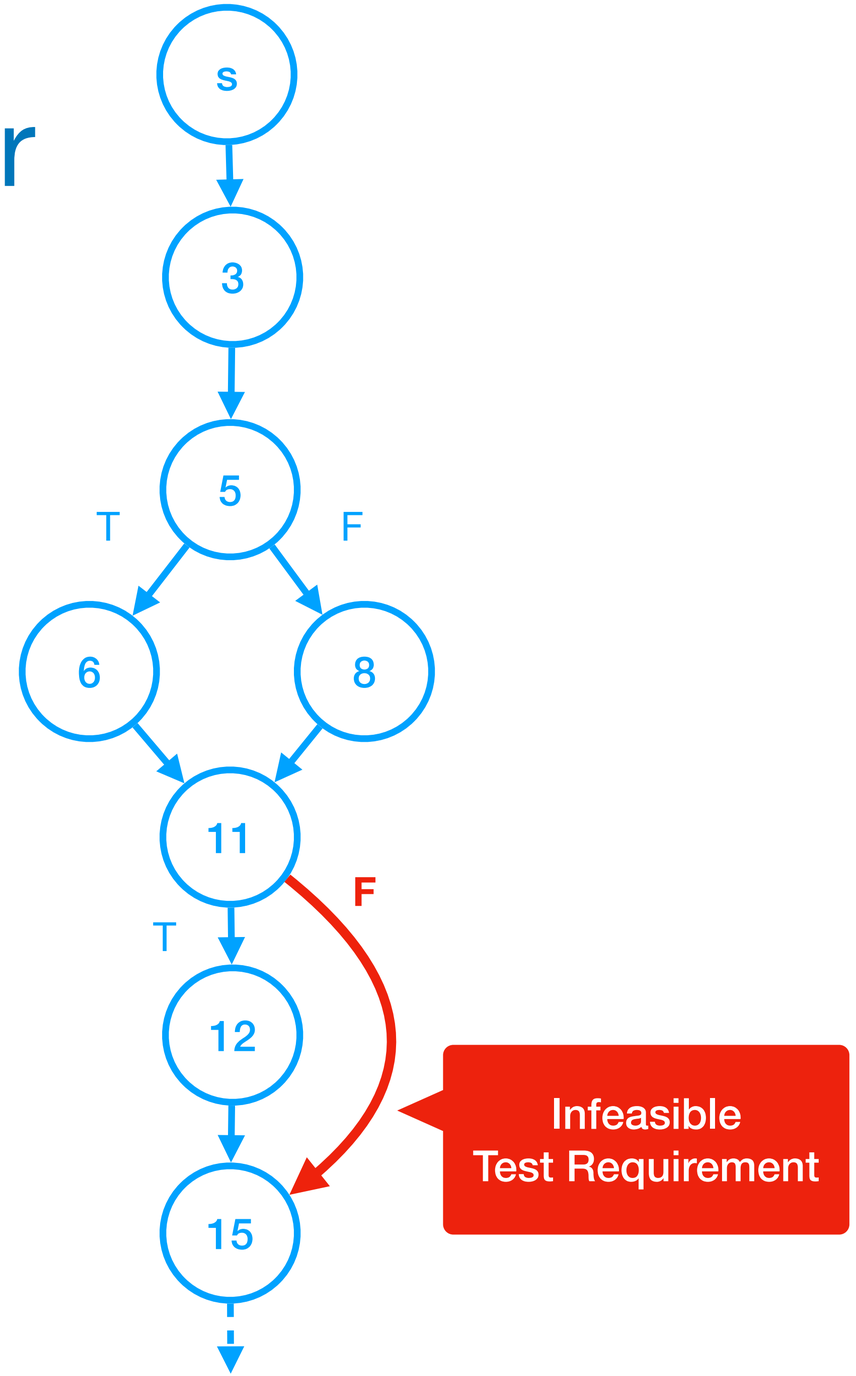
There is a problem lurking here for Branch Coverage. What is it?

```
1  public void testMe(int x) {
2
3    int y = 0;
4
5    if (x > 0) {
6        y = y + 1;
7    } else {
8        y = y + 2;
9    }
10
11   if (y > 0) {
12       y = y + 1;
13   }
14
15   // ...
```



There is a problem lurking here for Branch Coverage. What is it?

```
1  public void testMe(int x) {
2
3    int y = 0;
4
5    if (x > 0) {
6        y = y + 1;
7    } else {
8        y = y + 2;
9    }
10
11   if (y > 0) {
12       y = y + 1;
13   }
14
15   // ...
```



Infeasible Test Requirements

This **can** happen with Statement Coverage

- Infeasible statements point to **dead code**

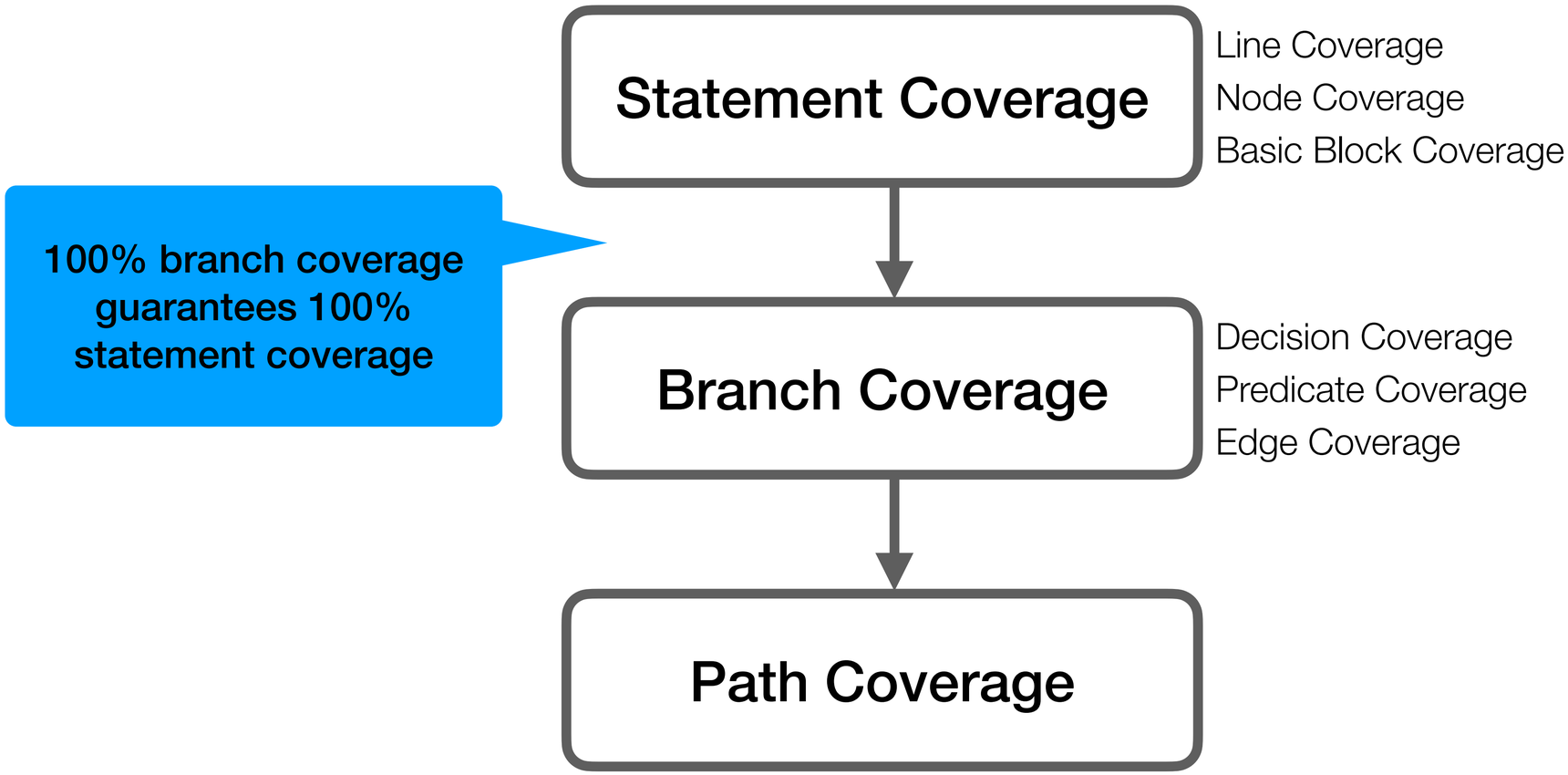
It is **more likely** to happen with Branch Coverage

- Infeasible branches point to redundant decisions in the code (e.g., defensive programming)

It is **very likely** with Path Coverage

- Infeasible paths are not necessarily the result of redundancy
- Not all paths through the CFG are legitimately possible in the actual code

Subsumption of Structural Coverage Criteria



When to Use Structural Coverage

Structural coverage level is a **useful metric** to understand how much of your code is executed by your test suite.

Common Rationale: You wouldn't want to release parts of your code that weren't exercised at least once by at least one test.

As such, Statement/Line Coverage is a **commonly used metric**

But Branch Coverage is **stronger**, and obtainable without much more additional effort

Path Coverage is less common and often intractable

White-Box Coverage Criteria based on Logic Analysis

Motivation for Logic Analysis

Consider the following defect in a method that checks if a user is allowed to vote in the UK (voters must be British and 18+):

```
if (nationality != "BRITISH" || age < 21)
    throw new Exception("Cannot vote”)
```

Branch coverage can be satisfied by testing only the sub-condition involving the nationality, i.e., the faulty sub-expression may **never determine the outcome** even if we test both outcomes of the branch.

Logic-based Coverage aims to test individual conditions in compound predicates.

```
public static int daysBetweenTwoDates(int year1, int month1, int day1,
                                       int year2, int month2, int day2) {
  ...
  if ((year2 < year1) ||
          (year2 == year1 && month2 < month1) ||
          (year2 == year1 && month2 == month1 && day2 < day1)) {
      int t = month2;
      month2 = month1;
      month1 = t;
      t = day2;
      day2 = day1;
      day1 = t;
      t = year2;
      year2 = year1;
      year1 = t;
  }
  ...
```

Disjunctions: $D_1 \vee D_2$

Disjunct 1

Disjunct 2

Disjunct 3

```
public static int daysBetweenTwoDates(int year1, int month1, int day1,
                                      int year2, int month2, int day2) {
  ...
  if ((year2 < year1) ||
      (year2 == year1 && month2 < month1) ||
      (year2 == year1 && month2 == month1 && day2 < day1)) {
      int t = month2;
      month2 = month1;
      month1 = t;
      t = day2;
      day2 = day1;
      day1 = t;
      t = year2;
      year2 = year1;
      year1 = t;
  }
  ...
```

Disjunct 1

Disjunct 2

Disjunct 3

```
public static int daysBetweenTwoDates(int year1, int month1, int day1,
                                      int year2, int month2, int day2) {
  ...
  if ((year2 < year1) ||
      (year2 == year1 && month2 < month1) ||
      (year2 == year1 && month2 == month1 && day2 < day1)) {
      int t = month2;
      month2 = month1;
      month1 = t;
      t = day2;
      day2 = day1;
      day1 = t;
      t = year2;
      year2 = year1;
      year1 = t;
  }
  ...
```

Branch Coverage does not ensure that each disjunct is exercised as true and false:

#Test	Inputs		Disjuncts			Branch Predicate
	year1/month1/day1	year2/month2/day2	1	2	3	
1	2026/12/13	2025/04/25	T	F	F	T
2	2025/04/25	2026/12/13	F	F	F	F

Analysing the Logic of a Predicate

Logical Operators

Operator	Math Symbol	Java
not	¬	!
and	∧	&&
or	∨	||

A **condition** is a boolean expression that is a component of a more complex predicate that does not contain any logical operators

```
if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
```

How many conditions are there in this if statement?

Analysing the Logic of a Predicate

Logical Operators

Operator	Math Symbol	Java
not	¬	!
and	∧	&&
or	∨	||

A **condition** is a boolean expression that is a component of a more complex predicate that does not contain any logical operators

```
if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
```

	Condition	Disjunct
1	year2 < year1	1
2	year2 == year1	2
3	month2 < month1	2
~~=2~~	~~year2 == year1~~	~~3~~
4	month2 == month1	3
5	day2 < day1	3

Condition Coverage

The most basic logic-based coverage criterion

Exercise **each condition** as **true** and **false**

```
...
if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
```

	Condition	Disjunct
1	year2 < year1	1
2	year2 == year1	2
3	month2 < month1	2
~~=2~~	~~year2 == year1~~	~~3~~
4	month2 == month1	3
5	day2 < day1	3

#Test	Inputs		Condition					Branch Predicate
	year1/month1/day1	year2/month2/day2	1	2	3	4	5	
1	2026/01/02	2025/01/01	T	F	F	T	T	T
2	2026/02/01	2026/01/01	F	T	T	F	F	T

While all conditions are evaluated to true and false, the branch predicate overall is only exercised as true.

Multiple Condition Coverage

More thorough than condition coverage

Exercise each possible **combination** of truth values for each condition

```
if ((year2 < year1) ||
        (year2 == year1 && month2 < month1) ||
        (year2 == year1 && month2 == month1 && day2 < day1)) {
```

	Condition	Disjunct
1	year2 < year1	1
2	year2 == year1	2
3	month2 < month1	2
~~=2~~	~~year2 == year1~~	~~3~~
4	month2 == month1	3
5	day2 < day1	3

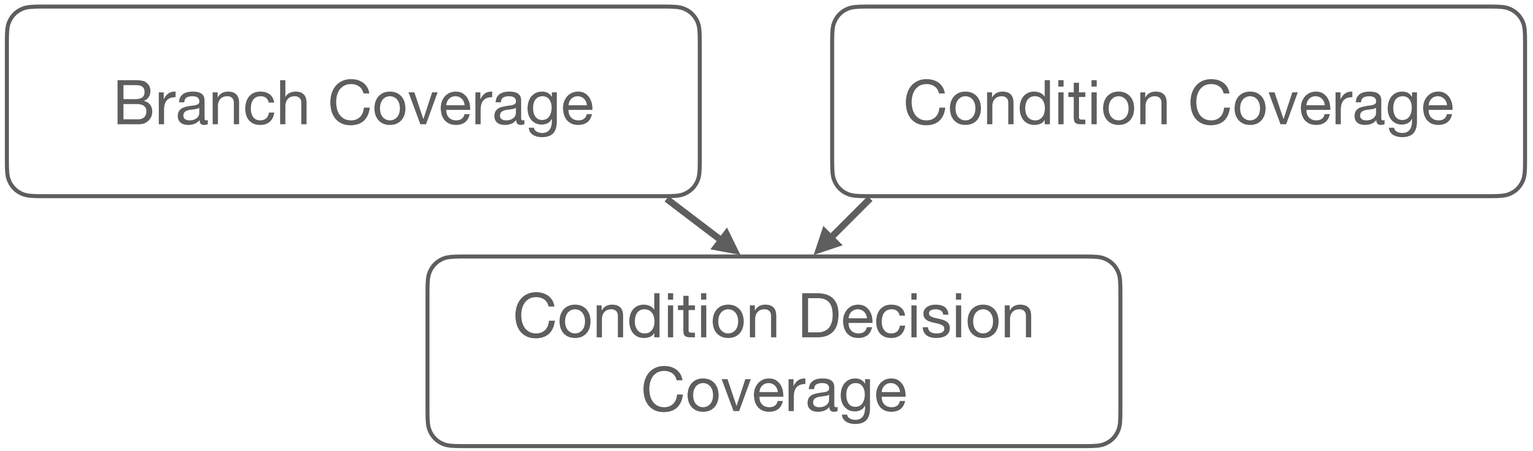
n conditions = 2^n test requirements

5 unique conditions = **2^5** = **32** test requirements

May seem like too many requirements to fulfil, but not all combinations are **feasible**, e.g., conditions 1 and 2 (resp. 3 and 4) cannot be true at the same time.

Condition Decision Coverage

Exercise **each condition** as true and false, as well as the overall **branch predicate**



Idea: **Combine branch and condition coverage** to address the issue that condition coverage **can be weaker than branch coverage**, while multiple condition coverage **may generate too many test requirements**.

MCDC: Modified Condition Decision Coverage

Multiple MCDC variants exist – this one is called Correlated MCDC

Exercise **each condition** as true and false.
Exercise **each decision** (branch predicate) as true and false.
Each **condition** **independently determines** the outcome of the decision.

	Condition	Disjunct
1	`year2 < year1`	1
2	`year2 == year1`	2
3	`month2 < month1`	2
~~=2~~	~~`year2 == year1`~~	~~3~~
4	`month2 == month1`	3
5	`day2 < day1`	3

Leads to **N+1** test requirements instead of 2^N, **avoiding combinatorial explosion**. In our example: **32→6**.

MCDC: Modified Condition Decision Coverage

Multiple MCDC variants exist – this one is called Correlated MCDC

c1 || (c2 && c3) || (c2 && c4 && c5)

Test suite of size **N+1=6**:

#Test	Input		Condition					Predicate
	Date 1	Date 2	c1 (y2 < y1)	c2 (y2 == y1)	c3 (m2 < m1)	c4 (m2 == m1)	c5 (d2 < d1)	
1	2026/05/10	2025/05/10	T	F	F	T	F	T
2	2026/05/10	2027/04/10	F	F	T	F	F	F
3	2026/05/10	2026/04/10	F	T	T	F	F	T
4	2026/05/10	2026/06/09	F	T	F	F	T	F
5	2026/05/10	2026/05/09	F	T	F	T	T	T
6	2026/05/10	2026/05/10	F	T	F	T	F	F

✅ Exercise **each condition** as true and false.
✅ Exercise **each decision** (branch predicate) as true and false.
✅ Each **condition** **independently determines** the outcome of the decision.

MCDC: Modified Condition Decision Coverage

Multiple MCDC variants exist – this one is called Correlated MCDC

Notice c3 and c4 also flip!
But this is **masked** by c2

c1 || (c2 && c3) || (c2 && c4 && c5)

Test suite of size **N+1=6**:

#Test	Input		Condition					Predicate
	Date 1	Date 2	c1 (y2 < y1)	c2 (y2 == y1)	c3 (m2 < m1)	c4 (m2 == m1)	c5 (d2 < d1)	
1	2026/05/10	2025/05/10	T	F	F	T	F	T
2	2026/05/10	2027/04/10	F	F	T	F	F	F
3	2026/05/10	2026/04/10	F	T	T	F	F	T
4	2026/05/10	2026/06/09	F	T	F	F	T	F
5	2026/05/10	2026/05/09	F	T	F	T	T	T
6	2026/05/10	2026/05/10	F	T	F	T	F	F

✅ Exercise **each condition** as true and false.
✅ Exercise **each decision** (branch predicate) as true and false.
✅ Each **condition** **independently determines** the outcome of the decision.

MCDC: Modified Condition Decision Coverage

Multiple MCDC variants exist – this one is called Correlated MCDC

c1 || (c2 && c3) || (c2 && c4 && c5)

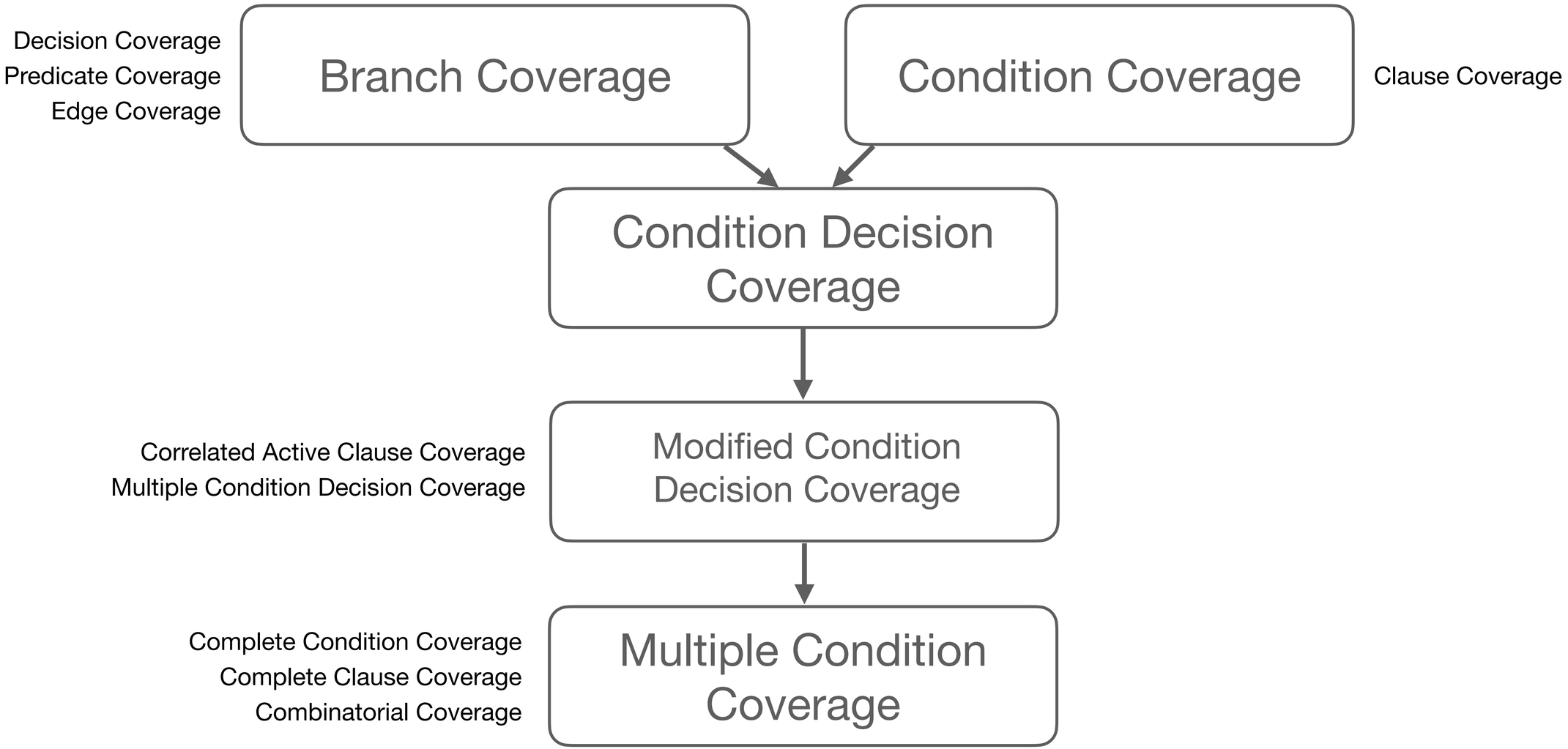
Test suite of size **N+1=6**:

#Test	Input		Condition					Predicate
	Date 1	Date 2	c1 (y2 < y1)	c2 (y2 == y1)	c3 (m2 < m1)	c4 (m2 == m1)	c5 (d2 < d1)	
1	2026/05/10	2025/05/10	T	F	F	T	F	T
2	2026/05/10	2027/04/10	F	F	T	F	F	F ○
3	2026/05/10	2026/04/10	F	T	T	F	F	T ○
4	2026/05/10	2026/06/09	F	T	F	F	T	F
5	2026/05/10	2026/05/09	F	T	F	T	T	T △
6	2026/05/10	2026/05/10	F	T	F	T	F	F △

✅ Exercise **each condition** as true and false.
✅ Exercise **each decision** (branch predicate) as true and false.
✅ Each **condition** **independently determines** the outcome of the decision.

Can you spot any other case of **masking**?

Subsumption Relationships



Consider this…

```
...
boolean yearSame = year2 == year1;
boolean yearAndMonthSame = yearSame && month2 == month1;

boolean secondDateBeforeFirstByYear = year2 < year1;
boolean secondDateBeforeFirstByMonth = yearSame && month2 < month1;
boolean secondDateBeforeFirstByDay = yearAndMonthSame && day2 < day1;

boolean secondDateBefore = secondDateBeforeFirstByYear ||
            secondDateBeforeFirstByMonth || secondDateBeforeFirstByDay;

if (secondDateBeforeFirst) {
    ...
}
...
```

When to use MCDC? MCDC is good for testing complex predicates. However, seasoned programmers will likely refactor their code to avoid such complex predicates in the first place.

MCDC and Testing Standards

ISO 26262 – “Road Vehicles – Functional Safety”
https://en.wikipedia.org/wiki/ISO_26262

EN 50128 – a functional safety standard used in the rail industry
https://www.adacore.com/industries/rail/en50128

DO-178B and DO-178C – “Software Considerations in Airborne Systems and Equipment Certification”
https://en.wikipedia.org/wiki/DO-178B
https://en.wikipedia.org/wiki/DO-178C

IEC 61508, a basic functional safety standard applicable to all industries. https://en.wikipedia.org/wiki/IEC_61508

Coverage Criteria:

The Pros and Cons of Coverage

The **“Cons”**

They can generate **too many** test requirements

Not all test requirements are **useful**

Higher coverage **doesn’t guarantee** that more bugs will be found

The **“Pros”**

Coverage criteria ensure that our test suites have certain properties or **strengths**

Coverage criteria tell us **how much** testing has been done and **when to stop testing**

Research on Coverage Criteria

The **relationship** between actual faults, failures and types of coverage and coverage levels

Where to concentrate testing to find faults and failures **earlier**

What the **ideal test suite size** is

How to check that a fault has already been repaired **and has not introduced further faults**

Can a fault be automatically repaired?

…